

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: K. Anusha

Roll No.: A24126552088

Date: 29-08-2026

1. TITLE

JavaScript Classes and DOM Manipulation – Dynamic Student Profile Generator

2. AIM

To design and develop a webpage that dynamically generates and displays a student profile using a JavaScript class, DOM manipulation, and event handling, based on details entered by the user through an HTML form.

3. OBJECTIVE

The objectives of this experiment are:

- To understand and define JavaScript classes and constructors.
 - To create objects from a class using the new keyword.
 - To select DOM elements using getElementById().
 - To handle button click events using addEventListener().
 - To validate user input before processing it.
 - To use the alert() function to display validation messages.
 - To dynamically create HTML elements using createElement().
 - To use appendChild() to insert dynamically created elements into the DOM.
 - To clear and update page content dynamically using innerHTML.
 - To style a form and a dynamically generated profile card using external CSS.
-

4. THEORY

A JavaScript class is a blueprint for creating objects with a defined constructor and properties. In this experiment, a **Student** class is created with a constructor that initializes the **name**, **rollNo**, **department**, and **cgpa** properties whenever a new Student object is created using the **new** keyword.

When the “Display Student Profile” button is clicked, JavaScript reads the values entered in the input fields, validates that none of them are empty, creates a new Student object from the entered values, and then dynamically builds and displays a profile card inside the page using DOM manipulation methods such as createElement() and appendChild().

In this experiment, HTML is used to create the structure of a page titled **Student Profile**, an external CSS file named **style.css** is used for styling, and an external JavaScript file named **script.js** is used to add interactivity.

The webpage consists of:

- **Registration Form:** Contains labelled input fields for Name, Roll Number, Department, and CGPA.
- **Display Student Profile Button:** Triggers input validation and profile generation when clicked.
- **Profile Container:** An initially empty `<div>` that JavaScript fills with a dynamically generated profile card showing the entered details.

CSS is used to style the container, form fields, button (with a hover effect), and the dynamically generated profile card, which is set apart with a distinct background colour and a left border.

The basic working flow is:

HTML Structure → CSS Styling → Class Definition → Event Handling → Input Validation → Object Creation → DOM Manipulation → Dynamic Profile Display

5. ALGORITHM / PROCEDURE

1. Create a new project folder for the webpage.
2. Create an HTML file named `index.html`.
3. Create an external CSS file named `style.css`.
4. Create an external JavaScript file named `script.js`.
5. Design a container with the heading "Student Registration".
6. Add labelled input fields for Name, Roll Number, Department, and CGPA.
7. Add a button labelled "Display Student Profile".
8. Add an empty `<div>` with `id="profileContainer"` to hold the generated profile.
9. Link `style.css` to style the form, button, and profile card.
10. Link `script.js` to add interactivity to the page.
11. In JavaScript, define a `Student` class with a constructor that sets `name`, `rollNo`, `department`, and `cgpa`.
12. Select the button and the profile container using `getElementById()`.
13. Add a click event listener to the button using `addEventListener()`.
14. Read the values entered in the Name, Roll Number, Department, and CGPA fields.
15. Validate that none of the fields are empty; if any field is empty, show an alert and stop.
16. Create a new `Student` object using the entered values.
17. Clear any previously displayed profile using `profileContainer.innerHTML = ""`.
18. Dynamically create a `<div>` with class `"profile"` to hold the new profile card.
19. Create an `<h2>` heading element for the profile card.
20. Create `<p>` elements for name, roll number, department, and CGPA, and set their text using `textContent`.

21. Append the heading and all detail paragraphs to the profile div.
 22. Append the profile div to the profileContainer using appendChild().
 23. Open index.html in a web browser and test the form with valid and invalid input.
-

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Student Profile</title>

  <!-- Link external CSS file -->
  <link rel="stylesheet" href="style.css">
</head>

<body>

  <div class="container">

    <h1>Student Registration</h1>

    <!-- Student input form -->
    <label for="name">Name:</label>
    <input type="text" id="name" placeholder="Enter student name">

    <label for="rollNo">Roll Number:</label>
    <input type="text" id="rollNo" placeholder="Enter roll number">

    <label for="department">Department:</label>
    <input type="text" id="department" placeholder="Enter department">

    <label for="cgpa">CGPA:</label>
    <input type="number" id="cgpa" step="0.01" placeholder="Enter CGPA">

    <!-- Button to generate the profile -->
    <button id="displayBtn">Display Student Profile</button>

    <!-- Dynamically generated profile will appear here -->
    <div id="profileContainer"></div>

  </div>

  <!-- Link external JavaScript file -->
  <script src="script.js"></script>

</body>
</html>
```

script.js

```

class Student {
    // Constructor initializes student properties
    constructor(name, rollNo, department, cgpa) {
        this.name = name;
        this.rollNo = rollNo;
        this.department = department;
        this.cgpa = cgpa;
    }
}

// Select the button and profile container
const displayButton = document.getElementById("displayBtn");
const profileContainer = document.getElementById("profileContainer");

// Add click event to the button
displayButton.addEventListener("click", function () {

    // Get values entered by the user
    const name = document.getElementById("name").value;
    const rollNo = document.getElementById("rollNo").value;
    const department = document.getElementById("department").value;
    const cgpa = document.getElementById("cgpa").value;

    // Check whether all fields are filled
    if (name === "" || rollNo === "" ||
        department === "" || cgpa === "") {

        alert("Please enter all student details.");
        return;
    }

    // Create an object of Student class
    const student = new Student(
        name,
        rollNo,
        department,
        cgpa
    );

    // Remove previously displayed profile
    profileContainer.innerHTML = "";

    // Create profile container dynamically
    const profile = document.createElement("div");
    profile.className = "profile";

    // Create heading
    const heading = document.createElement("h2");
    heading.textContent = "Student Profile";

    // Create student detail elements
    const namePara = document.createElement("p");
    namePara.textContent = "Name : " + student.name;

    const rollPara = document.createElement("p");
    rollPara.textContent = "Roll No : " + student.rollNo;

    const departmentPara = document.createElement("p");
    departmentPara.textContent =
        "Department : " + student.department;

    const cgpaPara = document.createElement("p");
    cgpaPara.textContent = "CGPA : " + student.cgpa;

    // Add elements to the profile
    profile.appendChild(heading);
    profile.appendChild(namePara);
    profile.appendChild(rollPara);
    profile.appendChild(departmentPara);
    profile.appendChild(cgpaPara);

    // Display the profile on the webpage
    profileContainer.appendChild(profile);
});

```

style.css

```

* {
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
  background-color: #a1939342;
  margin: 0;
  padding: 30px;
}

/* Main container */
.container {
  width: 450px;
  margin: auto;
  padding: 25px;
  background-color: rgb(98, 123, 124);
  border-radius: 10px;
  box-shadow: 0 4px 12px rgb(54, 104, 73);
}

/* Page heading */
h1 {
  text-align: center;
  color: #070707;
}

/* Form labels */
label {
  display: block;
  margin-top: 12px;
  font-weight: bold;
}

/* Input fields */
input {
  width: 100%;
  padding: 10px;
  margin-top: 5px;
  border: 1px solid #000000;
  border-radius: 5px;
}

/* Display button */
button {
  width: 100%;
  padding: 12px;
  margin-top: 20px;
  background-color: #1a1224;
  color: rgb(239, 240, 239);
  border: none;
  border-radius: 5px;
  font-size: 16px;
  cursor: pointer;
}

/* Button hover effect */
button:hover {
  background-color: #306ba9;
}

/* Dynamically generated student profile */
.profile {
  margin-top: 25px;
  padding: 20px;
  background-color: #f2ebf3;
  border-left: 5px solid #3d303d;
  border-radius: 8px;
}

/* Profile heading */
.profile h2 {
  margin-top: 0;
  text-align: center;
  color: #551c45;
}

/* Profile details */
.profile p {
  font-size: 16px;
  line-height: 1.6;
}

```

7. CODE EXPLANATION

Code / Concept	Explanation
class Student	Defines a class named Student.
constructor(name, rollNo, department, cgpa)	Initializes a new Student object with the given property values.

getElementById("displayBtn")	Selects the "Display Student Profile" button.
getElementById("profileContainer")	Selects the container element where the profile will be displayed.
addEventListener("click", fn)	Registers a function to run when the button is clicked.
.value	Reads the current value entered in an input field.
if (name == "" ...)	Validates that all required fields are filled before proceeding.
alert()	Displays a browser alert box containing a validation message.
new Student(...)	Creates a new object of the Student class using the entered values.
profileContainer.innerHTML = ""	Clears any previously displayed profile from the page.
document.createElement("div"/>"h2"/>"p")	Dynamically creates new DOM elements at runtime.
.className	Assigns the CSS class "profile" to the dynamically created div.
.textContent	Sets the visible text of a dynamically created heading or paragraph.
appendChild()	Attaches a newly created child element to its parent element.

8. TEST CASES AND EXECUTION DATA

The webpage was opened in a web browser and tested for input validation, dynamic profile generation, and correctness of the displayed data.

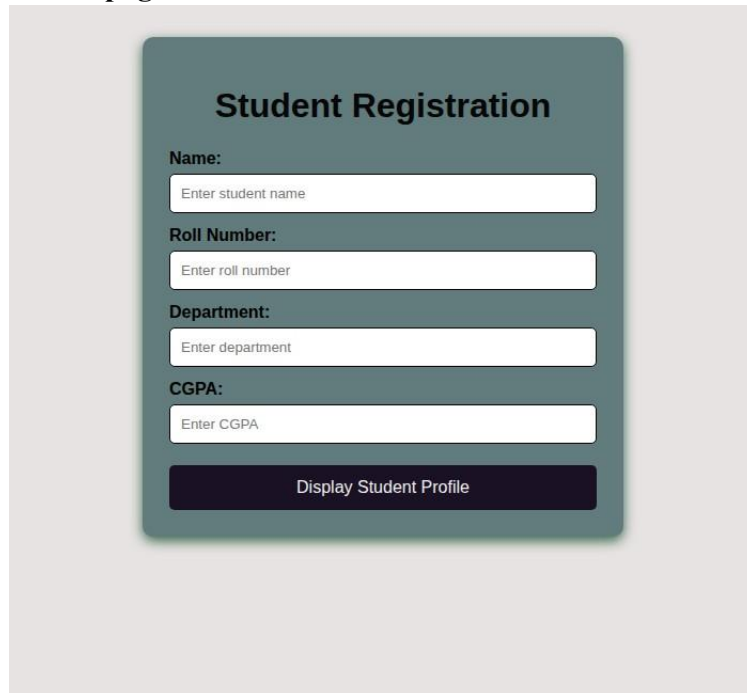
Test Case	Operation	Expected Result	Actual Result	Status
TC-01	Open index.html	Registration form should be displayed with an empty profile area	Displayed successfully	Pass
TC-02	Click "Display Student Profile" with all fields empty	Alert "Please enter all student details." should be displayed	Alert displayed with the expected message	Pass
TC-03	Fill only Name and click the button	Alert should be displayed since other fields are still empty	Alert displayed	Pass
TC-04	Fill Name, Roll Number, Department and CGPA correctly, then click the button	A student profile card should be displayed with the entered details	Profile card displayed correctly	Pass
TC-05	Click the button again after entering a different set of details	The previous profile should be replaced by the new profile	Profile updated correctly	Pass
TC-06	Enter a decimal CGPA value (e.g., 8.75)	CGPA field should accept decimal values	Decimal value accepted	Pass
TC-07	Enter a non-numeric value in the CGPA field	Browser should restrict the field to numeric/decimal input	Number input validated by browser	Pass

TC-08	Check the styling of the generated profile card	Profile card should be displayed with a distinct background and left border	Styled correctly	Pass
-------	---	---	------------------	------

9. OUTPUT

The webpage was executed in a web browser (Chromium). Screenshots of the actual page behaviour for the key test cases are shown below.

Output 1 (TC-01): Initial page load



Student Registration

Name:

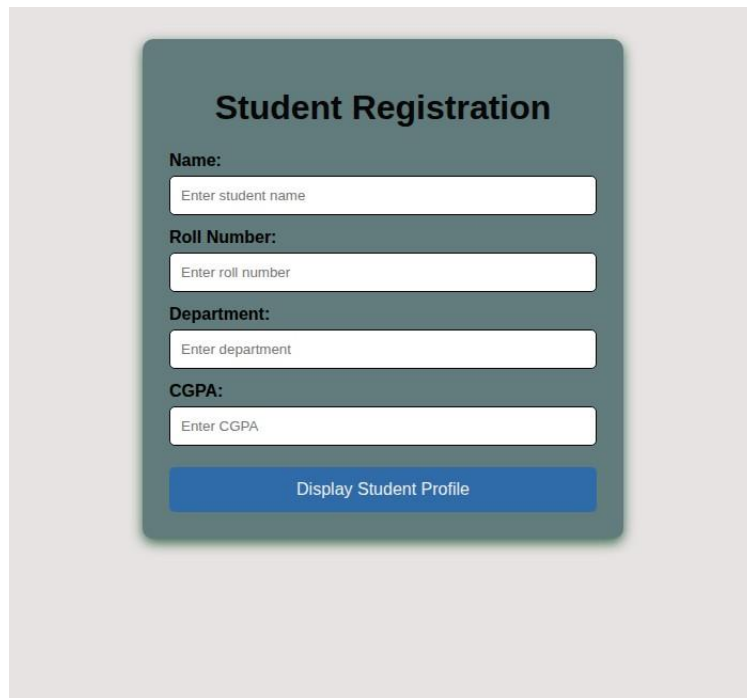
Roll Number:

Department:

CGPA:

Output 2 (TC-02): Clicking “Display Student Profile” with all fields empty triggers a browser alert box with the message:

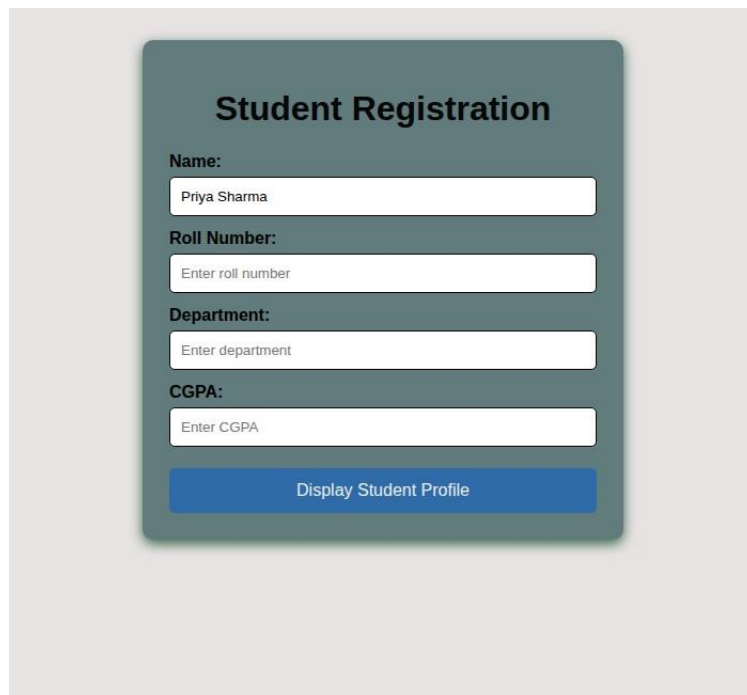
“Please enter all student details.”



A screenshot of a web form titled "Student Registration". The form is contained within a dark teal rounded rectangle. It has four input fields: "Name:" with placeholder "Enter student name", "Roll Number:" with placeholder "Enter roll number", "Department:" with placeholder "Enter department", and "CGPA:" with placeholder "Enter CGPA". Below these fields is a blue button labeled "Display Student Profile".

Output 3 (TC-03): With only the Name field filled, clicking the button again triggers the same alert:

“Please enter all student details.”



A screenshot of the same "Student Registration" form. The "Name:" field is now filled with the text "Priya Sharma". The other fields ("Roll Number:", "Department:", "CGPA:") still have their placeholder text. The blue "Display Student Profile" button remains at the bottom.

Output 4 (TC-04): Student profile displayed after valid input

Student Registration

Name:
Priya Sharma

Roll Number:
A24126552088

Department:
Computer Science

CGPA:
8.75

Display Student Profile

Student Profile

Name : Priya Sharma
Roll No : A24126552088
Department : Computer Science
CGPA : 8.75

Output 5 (TC-05): Profile replaced after entering a new student's details

Student Registration

Name:
Arjun Verma

Roll Number:
A24126552090

Department:
Electronics

CGPA:
9.10

Display Student Profile

Student Profile

Name : Arjun Verma
Roll No : A24126552090
Department : Electronics
CGPA : 9.10

10. RESULT

Thus, a webpage was successfully designed and developed to dynamically generate and display a student profile using a JavaScript class, DOM manipulation, and event handling. Input validation was implemented using conditional checks and the `alert()` function, a Student object was created using the class constructor, and the profile was dynamically built and displayed using `createElement()`, `textContent`, and `appendChild()`. The webpage was executed and verified successfully in a web browser, with all test cases producing the expected results.